

module-16-email-text

Chapter 16: Sending Email and Text Messages | 36 programs in learning order

Source: *Automate the Boring Stuff with Python 1e*, Al Sweigart (CC-BY-NC-SA). Syllabus Ch16: *smtplib*, *imapclient/imaplib*, *dues reminder*, *twilio SMS*, *chore mailer*, *email control*

How to use this guide (beginner path)

1) Read Concepts once. 2) Study the Visuals - redraw each on paper. 3) Follow the Program map. 4) Work programs 01..N in order. [shell]: type every plain line into IDLE, compare with # OUT: comments (errors shown are intentional demos). [script]: save, predict output, run with F5. 5) Drill Interview prep. 6) Teach it: explain each visual aloud without looking.

Concepts in detail

Email in code has two halves: SMTP SENDS (connect -> ehlo -> starttls -> login -> sendmail -> quit) and IMAP READS (login -> select folder -> search UIDs -> fetch -> parse addresses/body -> delete/quit). Capstones: scan a dues spreadsheet and mail only unpaid members; send SMS via Twilio; random chore assignment; and a computer that obeys emailed commands (with safeties).

Key ideas to carry forward

- `smtplib.SMTP(host, 587); .ehlo(); .starttls(); .login(u,p); .sendmail(f,t,msg); .quit()`.
- Search returns UIDs; fetch body with `[BODY[]]`; `pyzmail` parses addresses/subject/body cleanly.
- Delete = select folder -> store deleter flag -> expunge; disconnect cleanly.

Classic beginner mistakes

- ! Hard-coding passwords in scripts - use env vars/getpass and app passwords.
- ! Leaving test sends pointed at real people - dry-run on your own address first.

Study order hint: 01-08 SMTP send path, 09-23 IMAP read path, 24-28 dues reminder, 29-36 SMS/chores/remote-control.

Interview prep - questions you must answer cold

Say each answer aloud in under 60 seconds, then prove it with one program from this module.

Q1. SMTP send sequence?

A: SMTP(host,587) -> ehlo() -> starttls() -> login(u,p) -> sendmail(from,to,msg) -> quit(). TLS before login, always.

Q2. IMAP read sequence?

A: login -> select('INBOX') -> search() UIDs -> fetch(uid,'BODY[]') -> parse -> flag \Deleted + expunge -> logout().

Q3. How are addresses/bodies parsed?

A: Raw RFC822 needs a parser (pyzmail): get_addresses('from'), subject, text_part.get_payload().decode(charset).

Q4. Dues-reminder project logic?

A: openpyxl scan dues column, collect unpaid emails, sendmail() personalized bodies, log sent.

Q5. Twilio SMS send?

A: Client(sid, token).messages.create(to=..., from_=..., body=...) - credentials from env, test to your own number.

Q6. Email-controlled computer risks?

A: Anyone can spoof sender - whitelist addresses, require secret passphrases, confirm destructive commands, log everything.

Teach it to others

The one analogy that sticks

SMTP is the post office counter (you hand letters over); IMAP is your PO box you revisit (search, read, discard). Twilio is a courier for texts. Never tape your house key (password) to the letter - use env vars.

What your students will get wrong

- x Passwords in scripts are fine (use env/app passwords).
- x `sendmail to="` works for tests (send to yourself first).
- x Email From: proves identity (spoofable - verify + passphrase).

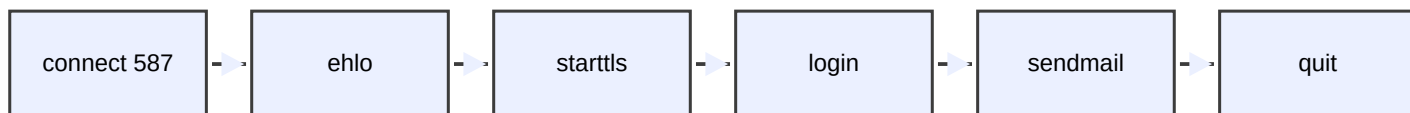
5-minute whiteboard plan

1. Draw the SMTP chain vs IMAP chain side by side.
2. Parse one raw message's subject line.
3. Students draft the filter condition.
4. Exit ticket: which call encrypts? (starttls).

Visualize it - draw these from memory

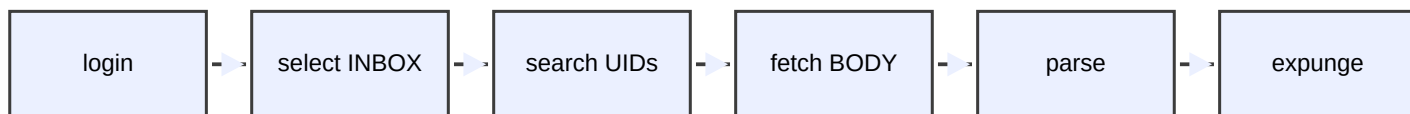
Redraw each figure on paper with labels before you touch code. If you cannot draw it, you cannot teach it.

Visual 1 - SMTP sends: the post-office chain



TLS BEFORE login, always. Test on your own address first; secrets in env, never in code.

Visual 2 - IMAP reads: revisit the PO box



Search returns IDs; fetch pulls bodies; flag Deleted + expunge removes. From: is spoofable - verify.

More examples - same idea, new clothes

Three bonus drills per module. Cover the output, predict, then run.

X1. Message builder

```
from email.mime.text import MIMEText
m = MIMEText('Hi Bo, dues due!')
m['Subject'] = 'Dues'
m['From'] = 'me@x.com'
m['To'] = 'bo@x.com'
print(m.as_string()[:60])
# EXPECTED: MIME headers + body preview
```

X2. Unpaid filter

```
rows = [('al', 1), ('bo', 0), ('cy', 0)]
print([n for n, paid in rows if not paid])
# EXPECTED: ['bo', 'cy']
```

X3. Reminder composer

```
def remind(name, amt):
    return f'Hi {name}, you owe ${amt}.'
print(remind('Bo', 25))
# EXPECTED: Hi Bo, you owe $25.
```

Program map - all 36 at a glance

```
01 01_sending_email.py
02 02_connecting_smtp_server.py
03 03_connecting_smtp_server_smtpobj.py
04 04_sending_smtp_hello.py
05 05_starting_tls_encryption.py
06 06_logging_smtp_server.py
07 07_sending_email_smtpobj.py
08 08_disconnecting_from_smtp.py
09 09_retrieving_deleting_emails.py
10 10_connecting_imap_server.py
11 11_logging_imap_server.py
12 12_searching_email.py
13 13_searching_email_imapobj.py
14 14_searching_email_uids.py
15 15_searching_email_imaplib.py
16 16_fetching_email_marking.py
17 17_fetching_email_marking_imapobj.py
18 18_getting_email_addresses.py
19 19_getting_email_addresses_message.py
20 20_getting_body_from.py
21 21_deleting_emails.py
22 22_disconnecting_from_imap.py
23 23_disconnecting_from_imap_imaplib.py
24 24_step_1_open.py
25 25_step_2_find.py
26 26_step_3_send.py
27 27_step_3_send_snip.py
28 28_step_3_send_sending.py
29 29_sending_text_messages.py
30 30_sending_text_messages_2.py
31 31_sending_text_messages_3.py
32 32_sending_text_messages_4.py
33 33_project_just_text.py
34 34_project_just_text_textmyself.py
35 35_random_chore_assignment.py
36 36_controlling_your_computer.py
```

Topic: Sending Email

01/36 01_sending_email.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import smtplib
smtpObj = smtplib.SMTP('smtp.example.com', 587)
smtpObj.ehlo()
# OUT: (250, b'mx.example.com at your service, [216.172.148.131]\nSIZE 35882577\
# OUT: n8BITIME\nSTARTTLS\nENHANCEDSTATUSCODES\nCHUNKING')
smtpObj.starttls()
# OUT: (220, b'2.0.0 Ready to start TLS')
smtpObj.login('bob@example.com', 'MY_SECRET_PASSWORD')
# OUT: (235, b'2.7.0 Accepted')
smtpObj.sendmail('bob@example.com', 'alice@example.com', 'Subject: So
# OUT: long.\nDear Alice, so long and thanks for all the fish. Sincerely, Bob')
# OUT: {}
smtpObj.quit()
# OUT: (221, b'2.0.0 closing connection ko10sm23097611pbd.52 - gsmtplib')
```

Topic: Connecting to an SMTP Server

02/36 02_connecting_smtp_server.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
smtpObj = smtplib.SMTP('smtp.gmail.com', 587)
```

```
type(smtpObj)
```

```
# OUT: <class 'smtplib.SMTP'>
```

03/36 03_connecting_smtp_server_smtpobj.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.
```

```
# Lines starting with >>> are input; other lines are expected output (commented).
```

```
smtpObj = smtplib.SMTP_SSL('smtp.gmail.com', 465)
```

Topic: Sending the SMTP "Hello" Message

04/36 04_sending_smtp_hello.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
smtpObj.ehlo()
```

```
# OUT: (250, b'mx.google.com at your service, [216.172.148.131]\nSIZE 35882577\
```

```
# OUT: n8BITIME\nSTARTTLS\nENHANCEDSTATUSCODES\nCHUNKING')
```

Topic: Starting TLS Encryption

05/36 05_starting_tls_encryption.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
smtpObj.starttls()
```

```
# OUT: (220, b'2.0.0 Ready to start TLS')
```

Topic: Logging in to the SMTP Server

06/36 06_logging_smtp_server.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
smtpObj.login(' my_email_address@gmail.com ', ' MY_SECRET_PASSWORD ')
```

```
# OUT: (235, b'2.7.0 Accepted')
```

Topic: Sending an Email

07/36 07_sending_email_smtpobj.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
smtpObj.sendmail(' my_email_address@gmail.com ', ' recipient@example.com ',  
# OUT: 'Subject: So long.\nDear Alice, so long and thanks for all the fish. Sincerely,  
# OUT: Bob')  
# OUT: {}
```

Topic: Disconnecting from the SMTP Server

08/36 08_disconnecting_from_smtp.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
smtpObj.quit()
```

```
# OUT: (221, b'2.0.0 closing connection ko10sm23097611pbd.52 - gsmtp')
```

Topic: Retrieving and Deleting Emails with IMAP

09/36 09_retrieving_deleting_emails.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.
# Lines starting with >>> are input; other lines are expected output (commented).

import imapclient
imapObj = imapclient.IMAPClient('imap.gmail.com', ssl=True)
imapObj.login(' my_email_address@gmail.com ', ' MY_SECRET_PASSWORD ')
# OUT: 'my_email_address@gmail.com Jane Doe authenticated (Success)'
imapObj.select_folder('INBOX', readonly=True)
UIDs = imapObj.search(['SINCE 05-Jul-2014'])
UIDs
# OUT: [40032, 40033, 40034, 40035, 40036, 40037, 40038, 40039, 40040, 40041]
rawMessages = imapObj.fetch([40041], ['BODY[]', 'FLAGS'])
import pyzmail
message = pyzmail.PyzMessage.factory(rawMessages[40041]['BODY[]'])
message.get_subject()
# OUT: 'Hello!'
message.get_addresses('from')
# OUT: [('Edward Snowden', 'esnowden@nsa.gov')]
message.get_addresses('to')
# OUT: [(Jane Doe', 'jdoe@example.com')]
message.get_addresses('cc')
# OUT: []
message.get_addresses('bcc')
# OUT: []
message.text_part != None
# OUT: True
message.text_part.get_payload().decode(message.text_part.charset)
# OUT: 'Follow the money.\r\n\r\n-Ed\r\n'
message.html_part != None
# OUT: True
message.html_part.get_payload().decode(message.html_part.charset)
# OUT: '<div dir="ltr"><div>So long, and thanks for all the fish!<br><br></div>-
# OUT: A<br></div>\r\n'
imapObj.logout()
```

Topic: Connecting to an IMAP Server

10/36 10_connecting_imap_server.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import imapclient
imapObj = imapclient.IMAPClient('imap.gmail.com', ssl=True)
```

Topic: Logging in to the IMAP Server

11/36 11_logging_imap_server.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
imapObj.login(' my_email_address@gmail.com ', ' MY_SECRET_PASSWORD ')
```

```
# OUT: 'my_email_address@gmail.com Jane Doe authenticated (Success)'
```

Topic: Searching for Email

12/36 12_searching_email.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import pprint
pprint.pprint(imapObj.list_folders())
# OUT: [(('\HasNoChildren',), '/', 'Drafts'),
# OUT:  (('\HasNoChildren',), '/', 'Filler'),
# OUT:  (('\HasNoChildren',), '/', 'INBOX'),
# OUT:  (('\HasNoChildren',), '/', 'Sent'),
# OUT:  --snip-
# OUT:  (('\HasNoChildren', '\Flagged'), '/', '[Gmail]/Starred'),
# OUT:  (('\HasNoChildren', '\Trash'), '/', '[Gmail]/Trash')]
```

13/36 13_searching_email_imapobj.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.
```

```
# Lines starting with >>> are input; other lines are expected output (commented).
```

```
imapObj.select_folder('INBOX', readonly=True)
```

14/36 14_searching_email_uids.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
UIDs = imapObj.search(['SINCE 05-Jul-2015'])  
UIDs  
# OUT: [40032, 40033, 40034, 40035, 40036, 40037, 40038, 40039, 40040, 40041]
```

15/36 15_searching_email_imaplib.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import imaplib
```

```
imaplib._MAXLINE = 10000000
```

Topic: Fetching an Email and Marking It As Read

16/36 16_fetching_email_marking.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.
Lines starting with >>> are input; other lines are expected output (commented).

```
rawMessages = imapObj.fetch(UIDs, ['BODY[]'])
import pprint
pprint.pprint(rawMessages)
# OUT: {40040: {'BODY[]': 'Delivered-To: my_email_address@gmail.com\r\n'
# OUT:                'Received: by 10.76.71.167 with SMTP id '
# OUT: --snip--
# OUT:                '\r\n'
# OUT:                '-----=_Part_6000970_707736290.1404819487066--\r\n',
# OUT:                'SEQ': 5430}}
```

17/36 17_fetching_email_marking_imapobj.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
imapObj.select_folder('INBOX', readonly=False)
```

Topic: Getting Email Addresses from a Raw Message

18/36 18_getting_email_addresses.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import pyzmail
message = pyzmail.PyzMessage.factory(rawMessages[40041]['BODY[]'])
```

19/36 19_getting_email_addresses_message.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
message.get_subject()  
# OUT: 'Hello!'  
message.get_addresses('from')  
# OUT: [('Edward Snowden', 'esnowden@nsa.gov')]  
message.get_addresses('to')  
# OUT: [(Jane Doe', 'my_email_address@gmail.com')]  
message.get_addresses('cc')  
# OUT: []  
message.get_addresses('bcc')  
# OUT: []
```

Topic: Getting the Body from a Raw Message

20/36 20_getting_body_from.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
message.text_part != None
```

```
# OUT:      True
```

```
message.text_part.get_payload().decode(message.text_part.charset)
```

```
# OUT:  'So long, and thanks for all the fish!\r\n\r\n-Al\r\n'
```

```
message.html_part != None
```

```
# OUT:      True
```

```
message.html_part.get_payload().decode(message.html_part.charset)
```

```
# OUT:  '<div dir="ltr"><div>So long, and thanks for all the fish!<br><br></div>-Al
```

```
# OUT:  <br></div>\r\n'
```

Topic: Deleting Emails

21/36 21_deleting_emails.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.
# Lines starting with >>> are input; other lines are expected output (commented).

imapObj.select_folder('INBOX', readonly=False)
UIDs = imapObj.search(['ON 09-Jul-2015'])
UIDs
# OUT:      [40066]
imapObj.delete_messages(UIDs)
# OUT: {40066: ('\Seen', '\Deleted')}
imapObj.expunge()
# OUT:      ('Success', [(5452, 'EXISTS')])
```

Topic: Disconnecting from the IMAP Server

22/36 22_disconnecting_from_imap.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
imapObj.logout()
```

23/36 23_disconnecting_from_imap_imaplib.py [script]

Script: save and run; predict the output first, then compare.

```
imaplib.abort: socket error: [WinError 10054] An existing connection was  
forcibly closed by the remote host
```

Topic: Step 1: Open the Excel File

24/36 24_step_1_open.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# sendDuesReminders.py - Sends emails based on payment status in spreadsheet.

import openpyxl, smtplib, sys

# Open the spreadsheet and get the latest dues status.
wb = openpyxl.load_workbook('duesRecords.xlsx')
sheet = wb.get_sheet_by_name('Sheet1')

lastCol = sheet.get_highest_column()
latestMonth = sheet.cell(row=1, column=lastCol).value

# TODO: Check each member's payment status.

# TODO: Log in to email account.

# TODO: Send out reminder emails.
```

Topic: Step 2: Find All Unpaid Members

25/36 25_step_2_find.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# sendDuesReminders.py - Sends emails based on payment status in spreadsheet.

--snip--

# Check each member's payment status.
unpaidMembers = {}
for r in range(2, sheet.get_highest_row() + 1):
    payment = sheet.cell(row=r, column=lastCol).value
    if payment != 'paid':
        name = sheet.cell(row=r, column=1).value
        email = sheet.cell(row=r, column=2).value
        unpaidMembers[name] = email
```

Topic: Step 3: Send Customized Email Reminders

26/36 26_step_3_send.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# sendDuesReminders.py - Sends emails based on payment status in spreadsheet.

--snip--

# Log in to email account.
smtpObj = smtplib.SMTP('smtp.gmail.com', 587)
smtpObj.ehlo()
smtpObj.starttls()
smtpObj.login(' my_email_address@gmail.com ', sys.argv[1])
```

27/36 27_step_3_send_snip.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# sendDuesReminders.py - Sends emails based on payment status in spreadsheet.

--snip--

# Send out reminder emails.
for name, email in unpaidMembers.items():
    body = "Subject: %s dues unpaid.\nDear %s,\nRecords show that you have not
paid dues for %s. Please make this payment as soon as possible. Thank you!" %
(latestMonth, name, latestMonth)
    print('Sending email to %s...' % email)
    sendmailStatus = smtpObj.sendmail('my_email_address@gmail.com', email, body)

    if sendmailStatus != {}:
        print('There was a problem sending email to %s: %s' % (email,
            sendmailStatus))
smtpObj.quit()
```

28/36 28_step_3_send_sending.py [script]

Script: save and run; predict the output first, then compare.

```
Sending email to alice@example.com...  
Sending email to bob@example.com...  
Sending email to eve@example.com...
```

Topic: Sending Text Messages

29/36 29_sending_text_messages.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
from twilio.rest import TwilioRestClient  
accountSID = 'ACxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx'  
authToken = 'xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx'  
twilioCli = TwilioRestClient(accountSID, authToken)  
myTwilioNumber = '+14955551234'  
myCellPhone = '+14955558888'  
message = twilioCli.messages.create(body='Mr. Watson - Come here - I want  
# OUT:    to see you.', from_=myTwilioNumber, to=myCellPhone)
```

30/36 30_sending_text_messages_2.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
message.to  
# OUT: '+14955558888'  
message.from_  
# OUT: '+14955551234'  
message.body  
# OUT: 'Mr. Watson - Come here - I want to see you.'
```

31/36 31_sending_text_messages_3.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
message.status  
# OUT: 'queued'  
message.date_created  
# OUT: datetime.datetime(2015, 7, 8, 1, 36, 18)  
message.date_sent == None  
# OUT: True
```

32/36 32_sending_text_messages_4.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
message.sid  
# OUT:      'SM09520de7639ba3af137c6fcb7c5f4b51'  
updatedMessage = twilioCli.messages.get(message.sid)  
updatedMessage.status  
# OUT:      'delivered'  
updatedMessage.date_sent  
# OUT:      datetime.datetime(2015, 7, 8, 1, 36, 18)
```

Topic: Project: "Just Text Me" Module

33/36 33_project_just_text.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# textMyself.py - Defines the textmyself() function that texts a message
# passed to it as a string.

# Preset values:
accountSID = 'ACxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx'
authToken = 'xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx'
myNumber = '+15559998888'
twilioNumber = '+15552225678'

from twilio.rest import TwilioRestClient

def textmyself(message):
    twilioCli = TwilioRestClient(accountSID, authToken)
    twilioCli.messages.create(body=message, from_=twilioNumber, to=myNumber)
```

34/36 34_project_just_text_textmyself.py [script]

Script: save and run; predict the output first, then compare.

```
import textmyself
textmyself.textmyself('The boring task is finished.')
```

Topic: Random Chore Assignment Emailer

35/36 35_random_chore_assignment.py [script]

Script: save and run; predict the output first, then compare.

```
chores = ['dishes', 'bathroom', 'vacuum', 'walk dog']
randomChore = random.choice(chores)
chores.remove(randomChore) # this chore is now taken, so remove it
```

Topic: Controlling Your Computer Through Email

36/36 36_controlling_your_computer.py [script]

Script: save and run; predict the output first, then compare.

```
qbProcess = subprocess.Popen(['C:\\Program Files (x86)\\qBittorrent\\  
qbittorrent.exe', 'shakespeare_complete_works.torrent'])
```